



Servidor de prueba APIs (Ejemplo listado de Clientes)





Contenido

1	Introducción.....	3
1.1	Estructura del proyecto	3
2	Instalar el mock.....	4
2.1	Instalación de json-server	4
2.2	Revisa el contenido del archivo package.json	4
2.3	Añadir el archivo de datos (db.json)	4
3	Arrancar la API	6
4	Endpoints que vamos a utilizar	6
5	Probar con Postman.....	7
5.4	GET — obtener todos	7
5.5	GET — obtener una empresa.....	7
5.6	7. POST — insertar empresa	7
5.7	PATCH — cambiar estado	8
5.8	DELETE — eliminar.....	8
5.9	Colección de Postman.....	8
6	Modificar nuestro modelo de datos (interface) en Angular.....	12
7	Modificar el Servicio (ClientesServices) de Angular	13
8	Modificar nuestro componente (lista-cliente) en Angular.	15
9	El flujo completo	18



1 Introducción

Este documento nos sirve de guía para montar un servidor de pruebas para nuestras llamadas a API's desde nuestra aplicación de Angular. A la vez que puedas probar exactamente las mismas operaciones desde Postman. **Angular → API mock → JSON**, y que puedas probar exactamente las mismas operaciones desde Postman.

Vamos a usar **json-server**, que genera automáticamente una API REST a partir de archivo con los datos **db.json**. La versión actual está en línea beta y requiere Node 22.12+, mientras que la rama estable es la 0.17.x que tiene una sintaxis clásica muy cómoda.

Para evitar problemas de compatibilidad con nuestros ejercicios, usaremos la versión **json-server 0.17.4**. En este documento se describe como instalar nuestro servidor de pruebas para las API's así como los cambios que tenemos que hacer en nuestro ejemplo de Angular (listado de clientes) para su adaptación

1.1 Estructura del proyecto

Crea una carpeta independiente de tu proyecto Angular (cualquier lugar de tu equipo):

```
api-clientes-mock/  
├── db.json  
├── package.json  
├── postman/  
│   └── clientes-api.postman_collection.json
```

El flujo quedará así:

```
Angular  
  │  
  │ HTTP  
  ▼  
http://localhost:3000  
  │  
  ▼  
json-server  
  │  
  ▼  
db.json
```

- **json-server** proporciona automáticamente las operaciones REST GET, POST, PUT, PATCH y DELETE para los recursos definidos como arrays en **db.json**. [GitHub](#)



2 Instalar el mock

Pasos para poner funcional el servidor json-server con Node 20.14:

2.1 Instalación de json-server

Crear un directorio donde vamos a instalar nuestro servidor de pruebas para las API's

```
mkdir api-clientes-mock
cd api-clientes-mock

npm init -y
npm install json-server@0.17.4
```

2.2 Revisa el contenido del archivo package.json

Crea o verifica el contenido del archivo package.json con el siguiente contenido:

```
{
  "name": "api-clientes-mock",
  "version": "1.0.0",
  "description": "API REST mock para practicar Angular HttpClient y Postman",
  "main": "index.js",
  "scripts": {
    "start": "json-server --watch db.json --port 3000"
  },
  "dependencies": {
    "json-server": "0.17.4"
  }
}
```

Después:

```
npm install
```

Y para arrancar:

```
npm start
```

La API estará disponible en:

```
http://localhost:3000
```

2.3 Añadir el archivo de datos (db.json)

Para que json-server pueda realizar el seguimiento de los diferentes elementos vamos a incluir un nuevo campo (**id**) como identificador técnico de la API y **código** como identificador de registro.

Copiar el siguiente contenido en el fichero (db.json) de nuestra carpeta del servidor API (api-clientes-mack) para inicializar un conjunto de datos con los que trabajar

```
{
  "clientes": [
    {
```



```
{
  "id": 1,
  "codigo": "COD-260001",
  "nombre": "Empresa 1",
  "nif": "2678984-A",
  "estado": "Activo",
  "direccion": {
    "calle": "Los Olmos",
    "numero": 24,
    "provincia": "Madrid"
  }
},
{
  "id": 2,
  "codigo": "COD-260002",
  "nombre": "Empresa 2",
  "nif": "2678984-B",
  "estado": "Activo",
  "direccion": {
    "calle": "Gran Via",
    "numero": 54,
    "provincia": "Madrid"
  }
},
{
  "id": 3,
  "codigo": "COD-260003",
  "nombre": "Empresa 3",
  "nif": "2678984-C",
  "estado": "Inactivo",
  "direccion": {
    "calle": "Serrano",
    "numero": 14,
    "provincia": "Madrid"
  }
},
{
  "id": 4,
  "codigo": "COD-260004",
  "nombre": "Empresa 4",
  "nif": "2678984-D",
  "estado": "Baja",
  "direccion": {
    "calle": "San Bernardo",
    "numero": 84,
    "provincia": "Madrid"
  }
},
{
  "id": 5,
  "codigo": "COD-ELIMINAR",
  "nombre": "Empresa Eliminar",
  "nif": "11111111-D",
  "estado": "Baja",
  "direccion": {
    "calle": "Goya",
    "numero": 84,
    "provincia": "Madrid"
  }
}
]
```



3 Arrancar la API

Utiliza la consola de comando y vete al directorio de mi servidor de APIs (api-clientes-mock). Allí ejecuta

```
npm start
```

Deberías ver algo parecido a:

```
JSON Server started
```

```
Resources
```

```
http://localhost:3000/clientes
```

```
Home
```

```
http://localhost:3000
```

Ahora puedes abrir en el navegador la siguiente dirección:

```
http://localhost:3000/clientes
```

y deberías tener el JSON de tus empresas.

4 Endpoints que vamos a utilizar

La API queda así:

Operación	Método	URL
Obtener todos	GET	/clientes
Obtener uno	GET	/clientes/1
Crear	POST	/clientes
Actualizar	PUT	/clientes/1
Cambiar estado	PATCH	/clientes/1
Eliminar	DELETE	/clientes/1

Por ejemplo:

```
GET http://localhost:3000/clientes
```

```
GET http://localhost:3000/clientes/3
```



5 Probar con Postman

Aquí se resumen diferentes ejemplos para usar la API desde Postman y verificar su funcionamiento

5.4 GET — obtener todos

GET `http://localhost:3000/clientes`
No necesitas Body.

5.5 GET — obtener una empresa

GET `http://localhost:3000/clientes/1`

Respuesta:

```
{
  "id": 1,
  "codigo": "COD-260001",
  "nombre": "Empresa 1",
  "nif": "2678984-A",
  "estado": "Activo",
  "direccion": {
    "calle": "Los Olmos",
    "numero": 24,
    "provincia": "Madrid"
  }
}
```

5.6 7. POST — insertar empresa

POST `http://localhost:3000/clientes`

Headers:

Content-Type: `application/json`

Body → raw → JSON:

```
{
  "codigo": "COD-260005",
  "nombre": "Empresa Nueva",
  "nif": "99999999-Z",
  "estado": "Activo",
  "direccion": {
    "calle": "Alcalá",
    "numero": 100,
    "provincia": "Madrid"
  }
}
```

`json-server` creará automáticamente el `id`. Las modificaciones mediante POST/PATCH/PUT/DELETE se persisten en el fichero JSON. GitHub



5.7 PATCH — cambiar estado

Por ejemplo, para poner la empresa 1 como Inactivo:

```
PATCH http://localhost:3000/clientes/1  
Body {"estado": " Inactivo "}
```

O para darla de baja:

```
PATCH http://localhost:3000/clientes/1  
Body {"estado": "Baja"}
```

Esto encaja directamente con nuestra función de cambiarEstado:

```
cambiarEstado(empresa: Empresa, nuevoEstado: EstadoCliente)
```

5.8 DELETE — eliminar

Para eliminar el cliente con id=5

```
DELETE http://localhost:3000/clientes/5
```

Esto eliminará: COD-ELIMINAR del db.json.

5.9 Colección de Postman

Puedes crear una colección llamada: API Clientes Mock con estas peticiones:

```
API Clientes Mock  
├── Clientes  
│   ├── GET - Obtener todos  
│   ├── GET - Obtener por ID  
│   ├── POST - Crear cliente  
│   ├── PATCH - Activar  
│   ├── PATCH - Desactivar  
│   ├── PATCH - Dar de baja  
│   └── DELETE - Eliminar
```

También puedes importar directamente esta colección:

```
{  
  "info": {  
    "_postman_id": "7f6e6e5a-3b4a-4d91-8e21-123456789001",  
    "name": "API Clientes Mock",  
    "description": "Colección para probar la API mock de clientes con json-server",  
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"  
  },  
  "variable": [  
    {  
      "key": "baseUrl",  
      "value": "http://localhost:3000"  
    },  
    {  
      "key": "clienteId",  
      "value": "5"  
    }  
  ]  
}
```



```
    "value": "1"
  }
],
"item": [
  {
    "name": "GET - Obtener todos",
    "request": {
      "method": "GET",
      "header": [],
      "url": {
        "raw": "{{baseUrl}}/clientes",
        "host": [
          "{{baseUrl}}"
        ],
        "path": [
          "clientes"
        ]
      }
    }
  },
  {
    "name": "GET - Obtener por ID",
    "request": {
      "method": "GET",
      "header": [],
      "url": {
        "raw": "{{baseUrl}}/clientes/{{clienteId}}",
        "host": [
          "{{baseUrl}}"
        ],
        "path": [
          "clientes",
          "{{clienteId}}"
        ]
      }
    }
  },
  {
    "name": "POST - Crear cliente",
    "request": {
      "method": "POST",
      "header": [
        {
          "key": "Content-Type",
          "value": "application/json"
        }
      ],
      "body": {
        "mode": "raw",
        "raw": "{\n  \"codigo\": \"COD-260005\",\n  \"nombre\": \"Empresa Nueva\",\n  \"nif\": \"99999999-Z\",\n  \"estado\": \"Activo\",\n  \"direccion\": {\n    \"calle\": \"Alcalá\",\n    \"numero\": 100,\n    \"provincia\": \"Madrid\"\n  }\n}"
      },
      "url": {
        "raw": "{{baseUrl}}/clientes",
        "host": [
          "{{baseUrl}}"
        ],
        "path": [
          "clientes"
        ]
      }
    }
  }
]
```



```
    ]
  }
},
{
  "name": "PATCH - Activar",
  "request": {
    "method": "PATCH",
    "header": [
      {
        "key": "Content-Type",
        "value": "application/json"
      }
    ],
    "body": {
      "mode": "raw",
      "raw": "{\n  \"estado\": \"Activo\"\n}"
    },
    "url": {
      "raw": "{{baseUrl}}/clientes/{{clienteId}}",
      "host": [
        "{{baseUrl}}"
      ],
      "path": [
        "clientes",
        "{{clienteId}}"
      ]
    }
  }
},
{
  "name": "PATCH - Desactivar",
  "request": {
    "method": "PATCH",
    "header": [
      {
        "key": "Content-Type",
        "value": "application/json"
      }
    ],
    "body": {
      "mode": "raw",
      "raw": "{\n  \"estado\": \"Inactivo\"\n}"
    },
    "url": {
      "raw": "{{baseUrl}}/clientes/{{clienteId}}",
      "host": [
        "{{baseUrl}}"
      ],
      "path": [
        "clientes",
        "{{clienteId}}"
      ]
    }
  }
},
{
  "name": "PATCH - Dar de baja",
  "request": {
    "method": "PATCH",
    "header": [
```



```
{
  "key": "Content-Type",
  "value": "application/json"
},
"body": {
  "mode": "raw",
  "raw": "{\n  \"estado\": \"Baja\"\n}"
},
"url": {
  "raw": "{{baseUrl}}/clientes/{{clienteId}}",
  "host": [
    "{{baseUrl}}"
  ],
  "path": [
    "clientes",
    "{{clienteId}}"
  ]
}
},
{
  "name": "DELETE - Eliminar",
  "request": {
    "method": "DELETE",
    "header": [],
    "url": {
      "raw": "{{baseUrl}}/clientes/{{clienteId}}",
      "host": [
        "{{baseUrl}}"
      ],
      "path": [
        "clientes",
        "{{clienteId}}"
      ]
    }
  }
}
]
```

Guárdalo como:

postman/clientes-api.postman_collection.json

y en Postman:

Import → clientes-api.postman_collection.json



6 Modificar nuestro modelo de datos (interface) en Angular

Ahora que tenemos el contrato de la API, tenemos que modificar nuestra definición de datos para incluir el nuevo campo (id). Para ello modificamos la interfaz en nuestro fichero cliente.model.ts

***NOTA:** En el código del ejemplo suministrado se ha creado otro interfaz (**ClienteAPI**) para mantener validos los componentes previos sin la utilización del servicio a través de la API.*

```
export enum EstadoCliente {
  Activo = 'Activo',
  Inactivo = 'Inactivo',
  Baja = 'Baja'
}

export interface ClienteAPI {
  id: number;           // Nuevo campo
  codigo: string;
  nombre: string;
  nif: string;
  estado: EstadoCliente;

  direccion: {
    calle: string;
    numero: number;
    provincia: string;
  };
}
```



7 Modificar el Servicio (ClientesServices) de Angular

El siguiente paso sera crear y/o modificar nuestro servicio para que utilice la API en la gestión de datos. (en lugar del array que ya no nos es necesario)

***NOTA:** Para mantener los ejemplos previos se ha creado un nuevo servicio `clientes.http.services.ts` en el que vamos a realizar esta implementación*

El servicio final (clientes.http.services.ts) en Angular quedaría c:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

import { ClienteAPI, EstadoCliente } from '../models/cliente.model';

@Injectable({
  providedIn: 'root'
})
export class ClientesService {

  private readonly http = inject(HttpClient);

  private readonly apiUrl = 'http://localhost:3000/clientes';

  obtenerListaClientes(): Observable<ClienteAPI[]> {
    return this.http.get<ClienteAPI[]>(this.apiUrl);
  }

  obtenerCliente(id: string): Observable<ClienteAPI> {
    return this.http.get<ClienteAPI>(`${this.apiUrl}/${id}`);
  }

  insertarCliente(cliente: ClienteAPI): Observable<ClienteAPI> {
    return this.http.post<ClienteAPI>(this.apiUrl, cliente);
  }

  cambiarEstado(cliente: ClienteAPI, nuevoEstado: EstadoCliente): Observable<ClienteAPI> {
    return this.http.patch<ClienteAPI>(`${this.apiUrl}/${cliente.id}`, {estado:
nuevoEstado});
  }

  activarEmpresa(empresa: ClienteAPI): Observable<ClienteAPI> {
    return this.cambiarEstado(empresa, EstadoCliente.Activo);
  }

  desactivarEmpresa(empresa: ClienteAPI): Observable<ClienteAPI> {
    return this.cambiarEstado(empresa, EstadoCliente.Inactivo );
  }

  bajaEmpresa(empresa: ClienteAPI): Observable<ClienteAPI> {
    return this.cambiarEstado(empresa, EstadoCliente.Baja);
  }

  eliminarEmpresa(empresa: ClienteAPI): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${empresa.id}`);
  }

}
```



Por último y no menos importante, tenemos que activar el HttpClient en nuestro fichero app-config.ts

```
import { ApplicationConfig } from '@angular/core';  
import { provideHttpClient } from '@angular/common/http';  
  
export const appConfig: ApplicationConfig = {  
  providers: [  
    provideHttpClient()  
  ]  
};
```



8 Modificar nuestro componente (lista-cliente) en Angular.

Al usar servicios con API nuestro código se va a volver asíncrono, esto significa que cuando ya hago una solicitud a la API las líneas que van después de la llamada se siguen ejecutando sin esperar a su respuesta

Las solicitudes al servicio en nuestros métodos se deben gestionar con subcribe y adaptar nuestro componente para trabajar con estos Observable, incluyendo loading, errores HTTP, refresco de la lista después de PATCH/DELETE y evitando suscripciones innecesarias.

NOTA: Estas modificaciones se han implementado en un nuevo componente (lista-clientes-final) para mantener los otros de ejemplo

El ejemplo final de nuestro componente tendría un aspecto como el siguiente

```
/*
lista-cliente-final.component : version final con servicio de API
- Objetivo:
- Cambio para uso de servicios con API (observables)
- Gestion de parametros de entrada y salida
*/
import { Component, output, inject } from '@angular/core';
import { NgClass, UpperCasePipe } from '@angular/common';

import { ClienteAPI, EstadoCliente } from '../models/cliente.model';
import { ClientesService } from '../services/clientes.http.service'

@Component({
  selector: 'app-lista-cliente-final',
  imports: [NgClass],
  templateUrl: './lista-cliente-final.component.html',
  styleUrls: ['./lista-cliente-final.component.css']
})
export class ListaClienteFinalComponent {

  public readonly estadoCliente = EstadoCliente; // Para poder usar el enumerado en la
  plantilla HTML

  // Gestion de comunicacion con el padre ->
  // @Output() empresaSeleccionada: Empresa | null = null; // version tradicional
  outEmpresaSeleccionada = output<ClienteAPI>();

  // #region - variables internas del componente
  filtrarBajas: boolean = false;
  empresaSeleccionada: ClienteAPI | null = null
  listaEmpresas: ClienteAPI[] = [];
  // #endregion

  // #region - inyección del servicio
  // tradicional
  // constructor(public clienteService: ClienteService) {}

  // recomendada en las ultimas versiones
  private servicioCientes = inject(ClientesService);
  // #endregion

  // empresaSeleccionada: Empresa | null = null;
```



```
ngOnInit(): void {
  this.getListasEmpresas();
}

// obtiene la lista empresa y selecciona opcionalmente la que tenga el codigo pasado
como parametro
getListasEmpresas(codigoSeleccionado?:string): void {
  this.servicioCientes.obtenerListaClientes().subscribe({
    next: (empresas) => {
      this.listaEmpresas = empresas;
      if (this.listaEmpresas.length === 0) {
        return;
      }
      if (codigoSeleccionado) {
        const empresa = this.listaEmpresas.find(empresa => empresa.codigo ===
codigoSeleccionado);
        if (empresa) { this.seleccionarEmpresa(empresa); }
      } else {
        this.seleccionarEmpresa(this.listaEmpresas[0]);
      }
    },
    error: (error) => {
      console.error('Error obteniendo empresas:', error);
    }
  });
}

seleccionarEmpresa(empresa: ClienteAPI) {
  // ejemplo de console.log / console.table /alert
  console.log('Empresa seleccionada:', empresa); // Muestra la informacion de la
lista de clientes en la consola
  console.table(empresa); // Muestra la lista clientes en la
consola en formato de tabla
  console.table(empresa, ['codigo', 'nombre']); // Filtrar: Muestra solo las
columnas 'codigo', 'nombre' de la tabla
  // seleccion de empresa
  this.empresaSeleccionada = empresa;
  this.outEmpresaSeleccionada.emit(empresa);
}

desactivarEmpresa(empresa: ClienteAPI): void {
  this.servicioCientes.desactivarEmpresa(empresa).subscribe({
    next: (empresaActualizada) => {
      console.log('Empresa desactivada:', empresaActualizada );
      this.getListasEmpresas();
    },
    error: (error) => {console.error('Error desactivando empresa:', error);
    }
  });
}

activarEmpresa(empresa: ClienteAPI): void {
  this.servicioCientes.activarEmpresa(empresa).subscribe({
    next: (empresaActualizada) => {
      console.log('Empresa activada:', empresaActualizada );
      this.getListasEmpresas();
    },
    error: (error) => {console.error('Error activando empresa:', error);
    }
  });
}

bajaEmpresa(empresa: ClienteAPI): void {
  this.servicioCientes.bajaEmpresa(empresa).subscribe({
    next: (empresaActualizada) => {
```



```
        console.log('Empresa dada de baja:', empresaActualizada );
        this.getListasEmpresas();
    },
    error: (error) => {console.error('Error dando baja empresa:',error);
    }
    });
}

cambiarEstado(empresa:ClienteAPI, newEstado:EstadoCliente): void {
    this.servicioCientes.cambiarEstado(empresa,newEstado);
}

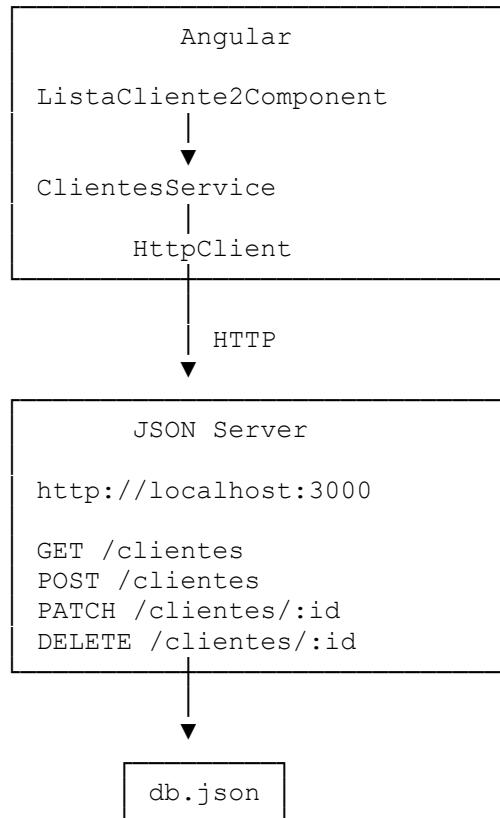
eliminarEmpresa(empresa: ClienteAPI): void {
    if (confirm(`¿Eliminar la empresa ${empresa.nombre}?`)) {
        this.servicioCientes.eliminarEmpresa(empresa).subscribe({
            next: () => {
                console.log('Empresa Eliminada:', empresa );
                this.getListasEmpresas();
            },
            error: (error) => {console.error('Error eliminando empresa:',error);
            }
        });
    }
}
```



9 El flujo completo

Ahora nuestro ejercicio tiene una arquitectura bastante más realista con el uso de API's mediante HTTP para comunicarse con un backend que es donde realmente se encuentran almacenados los datos

El esquema de funcionamiento ahora es



Además, **Postman y Angular atacan exactamente la misma API**, que es algo interesante en un modelo real donde podemos usar Postman para verificar las respuestas de las API's antes de pasar a la integración en nuestro código.

